

Nombre de la Universidad: Anáhuac Mayab

Campus: Merida

Nombre del Actividad de Aprendizaje: 3. Navegador web -
Documentación

Nombre del alumno: Emiliano Parra - Eduardo Téllez –
Santiago Loredó - Leonardo Argaez Ortega

Nombre del Docente: Lizbeth Hernández Olan

Fecha: 2/09/2026

Documentación del proyecto

Se debe realizar la siguiente documentación por cada proyecto.

Fase 1. Contextualización

- ¿Qué se debe hacer?

Para este proyecto se debe crear una aplicación en consola que simule el historial de un navegador web. El programa debe permitir visitar páginas escribiendo una URL, regresar a páginas anteriores con la opción Back, avanzar nuevamente con Forward, mostrar cuál es la página actual y salir del programa con Exit. También se debe limitar el historial de retroceso a 5 páginas y borrar el historial de avance cuando se visite una página nueva.

- ¿Qué se necesita?

Para realizar la aplicación se necesitan dos listas en Python: una para guardar las páginas a las que podemos regresar y otra para guardar las páginas a las que podemos avanzar. También se necesita establecer un límite de 5 páginas para el historial de retroceso y una variable para mostrar mensajes al usuario. Además, se utilizará un menú con opciones del 1 al 4 y una función para limpiar la pantalla después de cada acción.

- ¿Qué elementos se identifican?

Los principales elementos del programa son la clase NavegadorWeb, la página actual y las dos listas que guardan el historial. Para agregar páginas se utiliza `append()`, para quitar la última página se utiliza `pop()` y para borrar el historial de avance se utiliza `clear()`. También se cuenta con una función para limpiar la pantalla y otra llamada `main()`, que se encarga de mostrar el menú y recibir las opciones del usuario.

Fase 2. Definición de los requerimientos funcionales

Se debe elaborar el listado de los requerimientos del contexto planteado

Requerimientos	
Nombre	RF1 – Visitar nueva página web
Descripción	Permite ingresar una nueva URL. Si existe una página actual activa y la pila atrás (back_stack) no ha alcanzado el límite de 5 elementos, almacena la página actual en back_stack. Si la pila alcanza su límite, registra una advertencia de PILA LLENA. Actualiza la variable pagina_actual con la nueva dirección y vacía por completo la pila de avance (forward_stack.clear())
Entrada	Cadena de texto que representa la URL ingresada por el usuario (ejemplo: https://openai.com)
Salidas	Actualización de la página actual, limpieza de la pila de avance, almacenamiento en la pila atrás y mensaje de notificación en pantalla.
Nombre	RF2 – Retroceder a la página anterior (Back)
Descripción	Desplaza la página actual hacia la pila de avance (forward_stack) mediante una operación Push y asigna como nueva página actual el elemento extraído con Pop del tope de la pila atrás (back_stack). En caso de que la pila atrás se encuentre vacía, despliega la alerta "[ERROR 404] PILA VACÍA" sin alterar el estado.
Entrada	Selección de la opción 2 del menú interactivo.
Salidas	Transferencia del elemento entre pilas, actualización de la página activa o mensaje de error de pila vacía.
Nombre	RF3 – Avanzar a la página siguiente (Forward)
Descripción	Extrae el elemento superior de la pila de avance (forward_stack) y lo asigna a pagina_actual. Antes de realizar el avance, si la pila atrás no ha excedido el límite de capacidad, guarda la página activa previa en back_stack. Si la pila de avance está vacía, genera la notificación de error "[ERROR 404] PILA VACÍA".

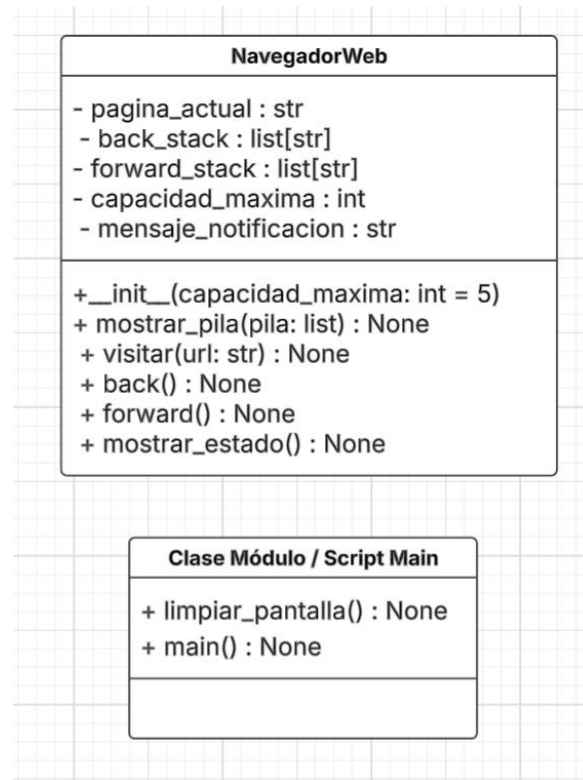
Entrada	Selección de la opción 3 del menú interactivo.
Salidas	Transferencia del elemento entre pilas, actualización de la página activa o mensaje de alerta en consola.

Nombre	RF4 – Mostrar el estado del navegador
Descripción	Imprime en la consola la estructura visual del navegador organizando en secciones la pila de avance, la pila de retroceso indicando el nivel de ocupación actual sobre el límite máximo ([ocupación/5]), la página web actual activa y los mensajes de notificación o errores generados en el último movimiento.
Entrada	Invocación desde el ciclo de ejecución o llamado al método mostrar_estado().
Salidas	Despliegue en texto estructurado de la consola con la información visual del estado del programa.

Nombre	RF5 – Control de interfaz y ciclo principal
Descripción	Controla la ejecución del programa a través de un ciclo continuo while True que limpia la pantalla con limpiar_pantalla(), imprime las opciones disponibles, valida que la URL ingresada no esté vacía, procesa las opciones del 1 al 4 y rompe el ciclo finalizando el programa al seleccionar la opción de salir
Entrada	Selección numérica del menú (1, 2, 3 o 4) e ingresos de cadenas de texto por teclado.
Salidas	Ejecución de los métodos de la clase, avisos de validación o la finalización del programa

Fase 3. Diseño

Determinar las clases que conforman el sistema, junto con los atributos y los métodos y crear el diagrama de clases UML, así como sus relaciones de herencia (si aplica).



https://lucid.app/lucidchart/b34ba5f6-d468-4f67-8112-0f8614b16cf8/edit?viewport_loc=-1442%2C-1034%2C1765%2C1028%2C0_0&invitationId=inv_dd0fc73f-bca8-4af6-92b8-4bd35104a525

Fase 4. Codificación

Se deben escribir los programas fuentes necesarios, basados en el diagrama de clase de la fase anterior:

- Implementación de los métodos (construcción de la parte lógica).
- Codificación de fórmulas
- Asignaciones
- Condicionales
- Documentación del código

Python

<https://onlinegdb.com/UGy373hFw>

```
import os

class NavegadorWeb:

    def __init__(self, capacidad_maxima=5):

        self.pagina_actual = "Ninguna (Inicio)"

        self.back_stack = []

        self.forward_stack = []

        self.capacidad_maxima = capacidad_maxima

        self.mensaje_notificacion = ""

    def mostrar_pila(self, pila):

        if not pila:

            print(" (Vacío)")

            return

        # Muestra desde el elemento superior (tope) hacia abajo

        for i, elem in enumerate(reversed(pila), 1):

            print(f" [{len(pila) - i + 1}] {elem}")

    def visitar(self, url):

        if self.pagina_actual and self.pagina_actual != "Ninguna (Inicio)":

            # Verificar si la pila atrás alcanzó su límite de capacidad

            if len(self.back_stack) >= self.capacidad_maxima:

                self.mensaje_notificacion = f"[!] Advertencia: PILA LLENA. Límite de {self.capacidad_maxima} páginas alcanzado."

            else:

                self.back_stack.append(self.pagina_actual)

                self.mensaje_notificacion = f"[!] Visitando nueva página: {url}"
```

```

else:

    self.mensaje_notificacion = f"[!] Visitando nueva página: {url}"

    self.pagina_actual = url

    # Al visitar una nueva página, la pila adelante se limpia por completo

    self.forward_stack.clear()

def back(self):

    if not self.back_stack:

        self.mensaje_notificacion = "[ERROR 404] PILA VACÍA: No hay páginas en la pila
atrás para retroceder."

        return

    self.forward_stack.append(self.pagina_actual)

    self.pagina_actual = self.back_stack.pop()

    self.mensaje_notificacion = f"[-] Retrocediendo a: {self.pagina_actual}"

def forward(self):

    if not self.forward_stack:

        self.mensaje_notificacion = "[ERROR 404] PILA VACÍA: No hay páginas en la pila
adelante para avanzar."

        Return

    if len(self.back_stack) >= self.capacidad_maxima:

        self.mensaje_notificacion = f"[!] Advertencia: PILA LLENA. La página no se
guardó en la pila atrás."

    else:

        self.back_stack.append(self.pagina_actual)

        self.mensaje_notificacion = f"[->] Avanzando a: {self.pagina_actual}"

def mostrar_estado(self):

    print("=" * 54)

```



```

print("          ESTADO DEL NAVEGADOR          ")

print("=" * 54)

# 1. Muestra la pila adelante primero

print(" PILA ADELANTE (Forward Stack):")

self.mostrar_pila(self.forward_stack)

print("-" * 54)

# 2. Muestra la pila atrás después

print(f" PILA ATRÁS (Back Stack)
[{len(self.back_stack)}/{self.capacidad_maxima}]:")

self.mostrar_pila(self.back_stack)

print("-" * 54)

# 3. Muestra la página actual al final

print(f" PÁGINA ACTUAL: {self.pagina_actual}")

print("=" * 54)

# Muestra notificaciones o errores del último movimiento

if self.mensaje_notificacion:

    print(f"\n{self.mensaje_notificacion}\n" + "-" * 54)

def limpiar_pantalla():

    # Ejecuta 'cls' en Windows o 'clear' en Linux/macOS

    os.system('cls' if os.name == 'nt' else 'clear')

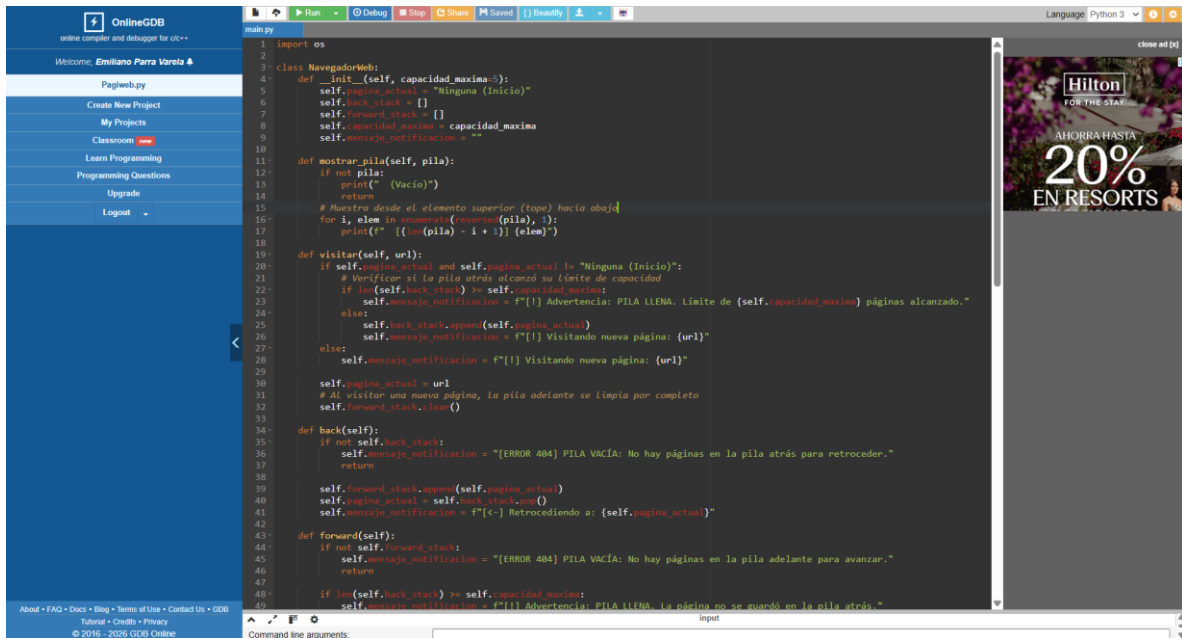
def main():

```

Fase 5. Pruebas.

Aquí se debe:

- Verificar que el programa cumpla con los requerimientos establecidos.
- Construcción de los documentos de entrega final.
- Adjuntar evidencias de resultados de la ejecución (impresión de pantallas).



```
1 import os
2
3 class NavegadorWeb:
4     def __init__(self, capacidad_maxima=5):
5         self.pagina_actual = "Ninguna (Inicio)"
6         self.back_stack = []
7         self.forward_stack = []
8         self.capacidad_maxima = capacidad_maxima
9         self.mensaje_notificacion = ""
10
11     def mostrar_pila(self, pila):
12         if not pila:
13             print(" (Vacío)")
14             return
15         # Muestra desde el elemento superior (top) hacia abajo
16         for i, elem in enumerate(reversed(pila), 1):
17             print(f" [{i}({pila[-i+1]}) {elem}"]
18
19     def visitar(self, url):
20         if self.pagina_actual and self.pagina_actual != "Ninguna (Inicio)":
21             # Verifico si la pila atrás alcanzó su límite de capacidad
22             if len(self.back_stack) >= self.capacidad_maxima:
23                 self.mensaje_notificacion = f"[!] Advertencia: PILA LLENA. Límite de {self.capacidad_maxima} páginas alcanzado."
24             else:
25                 self.back_stack.append(self.pagina_actual)
26                 self.mensaje_notificacion = f"[!] Visitando nueva página: {url}"
27         else:
28             self.mensaje_notificacion = f"[!] Visitando nueva página: {url}"
29
30         self.pagina_actual = url
31         # Al visitar una nueva página, la pila adelante se limpia por completo
32         self.forward_stack.clear()
33
34     def back(self):
35         if not self.back_stack:
36             self.mensaje_notificacion = "[ERROR 404] PILA VACÍA: No hay páginas en la pila atrás para retroceder."
37             return
38         self.forward_stack.append(self.pagina_actual)
39         self.pagina_actual = self.back_stack.pop()
40         self.mensaje_notificacion = f"< Retrocediendo a: {self.pagina_actual}"
41
42     def forward(self):
43         if not self.forward_stack:
44             self.mensaje_notificacion = "[ERROR 404] PILA VACÍA: No hay páginas en la pila adelante para avanzar."
45             return
46         self.pagina_actual = self.forward_stack.pop()
47         self.mensaje_notificacion = f"> Avanzando a: {self.pagina_actual}"
48
49     def __str__(self):
50         return f"Estado del navegador: {self.pagina_actual}, Back Stack: {self.back_stack}, Forward Stack: {self.forward_stack}, Mensaje: {self.mensaje_notificacion}"
```

Figura 5.1 Entorno de programación GDB



```
ESTADO DEL NAVEGADOR
=====
PILA ADELANTE (Forward Stack):
(Vacío)
=====
PILA ATRÁS (Back Stack) [0/5]:
(Vacío)
=====
PÁGINA ACTUAL: Ninguna (Inicio)
=====

Menú :
1. Visitar página (URL)
2. Retroceder (Back)
3. Avanzar (Forward)
4. Salir (Exit)

Selecciona una opción (1-4):
```

Figura 5.2 En ejecución

```
input
=====
ESTADO DEL NAVEGADOR
=====
PILA ADELANTE (Forward Stack):
(Vacio)
=====
PILA ATRÁS (Back Stack) [0/5]:
(Vacio)
=====
PÁGINA ACTUAL: Ninguna (Inicio)
=====

[ERROR 404] PILA VACÍA: No hay páginas en la pila adelante para avanzar.
=====

Menú :
1. Visitar página (URL)
2. Retroceder (Back)
3. Avanzar (Forward)
4. Salir (Exit)
Selecciona una opción (1-4): █
```

Figura 5.3 Cuando la pila de avance esté vacía, se mostrará el mensaje de error 404.

```
input
=====
ESTADO DEL NAVEGADOR
=====
PILA ADELANTE (Forward Stack):
(Vacio)
=====
PILA ATRÁS (Back Stack) [0/5]:
(Vacio)
=====
PÁGINA ACTUAL: hhttp/youtube
=====

[!] Visitando nueva página: hhttp/youtube
=====

Menú :
1. Visitar página (URL)
2. Retroceder (Back)
3. Avanzar (Forward)
4. Salir (Exit)
Selecciona una opción (1-4): 1
Ingresa la URL:
```

Figura 5.4 Opción 1

```
ESTADO DEL NAVEGADOR
=====
PILA ADELANTE (Forward Stack):
(Vacio)
=====
PILA ATRÁS (Back Stack) [5/5]:
[5] asfddgd
[4] asdaad
[3] asdaad
[2] asdaad
[1] hhttp/youtube
=====
PÁGINA ACTUAL: asfasffa
=====

[!] Advertencia: PILA LLENA. Limite de 5 páginas alcanzado.
=====

Menú :
1. Visitar página (URL)
2. Retroceder (Back)
3. Avanzar (Forward)
4. Salir (Exit)

Selecciona una opción (1-4): █
```

Figura 5.5 Cuando se llena la opción 1.

```
ESTADO DEL NAVEGADOR
=====
PILA ADELANTE (Forward Stack):
[3] asdaad
[2] asfddgd
[1] 123
=====
PILA ATRÁS (Back Stack) [3/5]:
[3] asdaad
[2] asdaad
[1] hhttp/youtube
=====
PÁGINA ACTUAL: asdaad
=====

[->] Avanzando a: asdaad
=====

Menú :
1. Visitar página (URL)
2. Retroceder (Back)
3. Avanzar (Forward)
4. Salir (Exit)

Selecciona una opción (1-4): █
```

Figura 5.6 Cuando se comparten las listas